**Сетевой уровень L3 (Network Layer):**

- **Функции**: отвечает за маршрутизацию пакетов данных между узлами сети. Обеспечивает логическую адресацию и определяет пути передачи данных через различные сети.
- **Примеры**: IP-адресация, маршрутизаторы, IP-протокол.
- **Необходимые для этого уровня понятия:** IP, ARP, IPsec, ICMP, IGMP, OSPF.

Сетевой уровень

Если совсем просто, на этом уровне происходит передача информации между различными локальными сетями с помощью таких устройств, как маршрутизатор. Эти устройства не обрабатывают данные выше L3 уровня и передают их с помощью определенных алгоритмов между разными подсетями.

**Транспортный уровень L4 (Transport Layer):**

- **Функции**: обеспечивает надежную передачу данных, контроль ошибок, сегментацию и повторную сборку данных. Гарантирует, что данные передаются без потерь и дубликатов.
- **Примеры**: TCP (Transmission Control Protocol), UDP (User Datagram Protocol).
- **Необходимые для этого уровня понятия:** TCP, UDP, SCTP, SSL, TLS.

Транспортный уровень

Если совсем просто, на этом уровне информация передается по сети с помощью протоколов уровня L4. Например, это могут быть файлы с вашего компьютера или потоковое видео, которое вы смотрите на видео хостинге.

| TCP/IP | OSI | Пример |
|---|---|---|
| Прикладной | Прикладной | HTTP |
| | Представительский | |
| | Сеансовый | |
| Транспортный | Транспортный | TCP, UDP |
| Сетевой | Сетевой | IP |
| Канальный | Канальный | Ethernet |
| | Физический | витая пара |

| Тип поля | Поведение |
|---|---|
| primitive | `int b = 42;` → 4 байта |
| String | UTF-8 + длина |
| Serializable | Рекурсивная сериализация |
| `transient` | Игнорируется (`null` после десериализации) |
| static/final | Игнорируются (принадлежат классу) |

Nio.buffer.compact – смещение недочитанного в начало

---

---

DECORATOR

```
class PicklePancake extends Pancake {
  Pancake base;
  PicklePancake(Pancake p) { base = p; }
  double getPrice() { return base.getPrice() + 30); }
}
```

- Фабричный метод
  - Statement stat = connection.createStatement()

```
class Creator {
  protected abstract Product getProduct();
}

class Creator1 {
  public Product getProduct() {
    return new Product1();
  }
}
```



## Factory Method

- Убирает зависимость от конкретных продуктов
- Упрощает добавление новых продуктов
- На каждый продукт нужен свой создатель

```
class Client {
    void deliverCargo(boolean isSea) {
        if (isSea) new Ship().deliver();     // Зависимость!
        else new Truck().deliver();
    }
}

// ✅ После паттерна
class Client {
    void deliverCargo(Logistics logistics) {
        logistics.planDelivery();  // НИКАКИХ if/else!
    }
}
```

**Клиент не знает о конкретных классах** — только интерфейс.

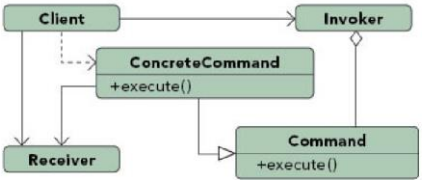
2. OCP — **расширение без изменений**

```java
// Новый продукт? Просто новый креатор!
class AirLogistics extends Logistics {
    Transport createTransport() { return new Plane(); }
}

// Клиентский код НЕ МЕНЯЕТСЯ
logistics = new AirLogistics();
logistics.planDelivery();  // "Самолёт"
```

## Шаблон Команда (Command)

- Управление командами
- Разделение вызова и исполнения команд
- Можно организовать очередь команд и макрокоманды



## Flyweight

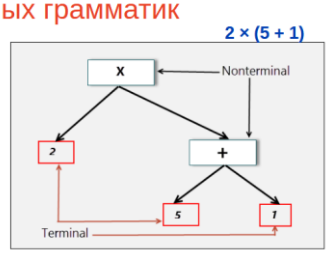
```
class TreeType {
  Image image;
  Color color;
}
```

```
class Tree {
  int x, y;
  TreeType type;
}
```

```
TreeType birch = new TreeType(Color.GREEN, 'birch.png');
TreeType pine = new TreeType(Color.GREEN, 'pine.png');
Tree t1 = new Tree(15,35,birch);
Tree t1 = new Tree(27,31,pine);
Tree t1 = new Tree(20,40,birch);
Tree t1 = new Tree(32,14,pine);
```

## Interpreter

- Простота расширения и изменения языка
- Простота добавления новых способов
- Сложность сопровождения сложных грамматик




## Singleton

```
class Singleton {
  private final static Singleton INSTANCE = new Singleton();

  private Singleton() { }

  public static Singleton getInstance() {
    return INSTANCE;
  }
}
```

Singleton

```
class Taxi implements Strategy {
  void findRoute() {
    allowedStart(ROAD);
    allowedFinish(ROAD);
    setTariff(BY_KM);
    setTime(now() + 15*60);
    navigate();
}
```

```
class Walk implements Strategy {
  void findRoute() {
    allowedStart(ANY);
    allowedFinish(ANY);
    setTariff(ZERO);
    setTime(now());
    navigate();
}
```

```
class Bus implements Strategy {
  void findRoute() {
    allowedStart(STOPS);
    allowedFinish(STOPS);
    setTariff(FIXED);
    setTime(getTimeTable());
    navigate();
}
```

```
Strategy st = getStrategy();
st.findRoute();
```

## Adapter

- Адаптирует один интерфейс к другому

```
class EUPlug {
  connect(EUSocket eus) {
    eus.getEUPower();
  }
}
```

```
class Adapter implements EUSocket {
  UKSocket uks;
  Adapter(UKSocket s) { uks = s; }
  getEUPower() { uks.getUKPower(); }
}
```

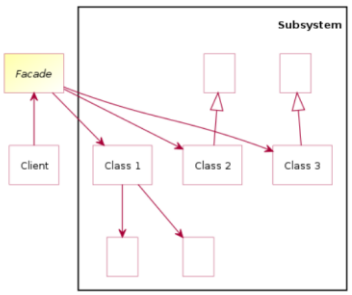
```
interface UKSocket {
  getUKPower()
}
```

```
EUPlug myPlug = myDevice.getPlug();
UKSocket roomSocket = room.getSocket();
EUSocket adapter = new Adapter(roomSocket);
myPlug.connect(adapter);
```

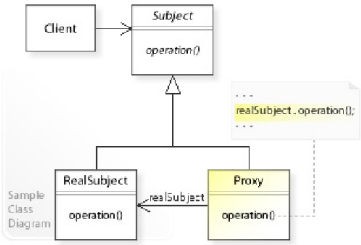
## Facade

- Фасад - простой интерфейс к сложным системам
- Система может меняться
- Доступ через фасад

```
class Facade {
  public go() {
    c0.connect(c1,c2,c3,c42);
    c10.activate(c11,c42,c6);
    c24.switch(c5,c3,c2,c42);
    c999.check(c7,c42,c1,c2);
    c0.start(c42,c2025,c999);
  }
}
```




- Заместитель

```
class Proxy implements Subject {
  RealSubject real;
  public request() {
    if (real == null) {
      real = new RealSubject();
    }
    return real.request();
  }
}
```

```
Subject subject = new Proxy();
subject.request();
```